

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Software (rediseño)

Práctica Experimental — Unidad IV

Temas 13–16: Arquitecturas Web, APIs REST,
SOAP y Tendencias Emergentes

Asignatura: Aplicaciones Web — 5.^{to} nivel, paralelo A

Docente: Guerrero Ulloa Gleiston Cicerón

Grupo: PFC — Pallo · Beltrán · Cruz

PFC: SGED — Sistema de Gestión Estudiantil Deportiva

Período académico: 2026–2027 PPA

10 de agosto de 2026

Resumen

El presente informe documenta la Práctica Experimental correspondiente a la Unidad IV de la asignatura Aplicaciones Web, abarcando los Temas 13 a 16. El equipo diseñó, implementó y demostró el **Sistema de Gestión Estudiantil Deportiva (SGED)**, una aplicación web completa basada en el patrón Modelo–Vista–Controlador (MVC) con backend en *Spring Boot 3.2* (Java 21) y frontend en *Angular 21*. Se expone la API REST del sistema con documentación interactiva mediante Swagger UI accesible en `/api/docs`, se ejecutan en vivo al menos tres endpoints representativos y se elabora una comparativa rigurosa entre SOAP y REST con nueve criterios de análisis. Finalmente, se reflexiona críticamente sobre las tendencias emergentes: Jamstack, Progressive Web Apps (PWA) e Inteligencia Artificial generativa aplicada al desarrollo web.

Abstract. This report documents the Experimental Practice for Unit IV of the Web Applications course, covering Topics 13–16. The team designed, implemented, and demonstrated the **Sports Student Management System (SGED)**, a full-stack web application built on the Model–View–Controller (MVC) pattern using a *Spring Boot 3.2* (Java 21) backend and an *Angular 21* frontend. The system’s REST API is presented with interactive documentation via Swagger UI at `/api/docs`; at least three representative endpoints are executed live. A rigorous nine-criterion comparison between SOAP and REST is provided, and the report closes with a critical reflection on emerging trends: Jamstack, Progressive Web Apps (PWA), and generative Artificial Intelligence in web development.

Palabras clave: REST, SOAP, MVC, API, OpenAPI, Swagger, Jamstack, PWA, IA generativa, Spring Boot.

Keywords: REST, SOAP, MVC, API, OpenAPI, Swagger, Jamstack, PWA, generative AI, Spring Boot.

Índice

Resumen	1
1 Introducción	5
2 Revisión Bibliográfica	5
2.1 Fielding (2000) — REST como estilo arquitectónico	5
2.2 Richardson y Ruby (2007) — REST Práctico	6
2.3 SpringDoc OpenAPI 2 — Documentación oficial	6
2.4 W3C (2007) — Especificación SOAP 1.2	7
2.5 Masse (2011) — REST API Design Rulebook	7
2.6 Garriga et al. (2021) — Adopción de APIs en América Latina	7
2.7 Fielding y Reschke (2014) — RFC 7231: Semántica HTTP/1.1	8
2.8 WCAG 2.2 (W3C, 2023) — Pautas de Accesibilidad para el Contenido Web	8
2.9 ISO/IEC 25010:2023 — Modelo de calidad de producto de software	9
2.10 Evans (2003) y Kleppmann (2017) — Consistencia de datos relacionados	9
3 El Patrón MVC y su Implementación en Frameworks Modernos	10
3.1 Historia y evolución del MVC	10
3.2 El patrón Front Controller	11
3.3 Comparativa de frameworks MVC	12
3.4 Justificación del framework del PFC	12
4 Aplicación MVC Completa — SGED	13
4.1 Descripción general	13
4.2 Arquitectura MVC	13
4.3 Módulos verificados	14
4.4 Verificación en laboratorio	14
5 Diseño de la API REST: Principios de Fielding, JWT y OpenAPI	17
5.1 Los seis principios de Fielding en el diseño de la API	17
5.2 Diseño de URIs: reglas aplicadas y anti-patronos detectados	17

5.3	JWT: estructura, flujo y revocación	19
5.4	OpenAPI 3 como contrato: Swagger UI y Redoc	20
5.5	Configuración de Swagger UI	21
5.6	Endpoints demostrados en vivo	21
5.6.1	Endpoint 1 — POST /api/auth/login	21
5.6.2	Endpoint 2 — GET /api/categorias/activas	22
5.6.3	Endpoint 3 — POST /api/asistencias/qr/sesion/{idSesion}/token	22
5.7	Checklist de ensayo previo al laboratorio	23
6	Comparativa SOAP vs. REST	23
6.1	Tabla de criterios	23
6.2	Ejemplo SOAP y su equivalente REST	23
6.2.1	SOAP — NumberToWords (servicio público verificado con curl)	23
6.2.2	REST — Equivalente en SGED	25
6.3	Análisis del contraste	25
6.4	Sobre el formato de respuesta de la API	26
7	Consumo de Servicios Web Externos	26
7.1	La API externa elegida y por qué	26
7.2	Cliente HTTP del lado del servidor	27
7.3	Caché de respuestas externas: patrón cache aside	27
7.4	Manejo de errores y degradación	28
7.5	Verificación y métricas	28
7.6	Presentación en la interfaz	29
8	Tendencias Emergentes en el Desarrollo Web	30
8.1	Jamstack: desacoplamiento como paradigma de arquitectura	30
8.2	Progressive Web Apps (PWA): cierre de la brecha web/nativa	30
8.2.1	Demo funcional — Service Worker en el SGED	31
8.3	Inteligencia Artificial Generativa: reconfiguración del rol del desarrollador	32
9	Seguridad, Pruebas de Rendimiento y Despliegue	33
9.1	Auditoría de seguridad OWASP	33
9.2	Prueba de carga con Apache Bench	34

9.3	Containerización con Docker Compose	34
9.4	Reflexión sobre el proceso de aprendizaje	35
10	Conclusiones	36
11	Trabajo Futuro	37
	Referencias	38
A	Anexo C — Preguntas de Análisis	39
A.1	La decisión técnica más difícil (Bloom 6 — Evaluación)	40
A.2	Escalar a 500 usuarios concurrentes con 0 % de error (Bloom 5 — Síntesis)	40
A.3	Los 6 principios REST de Fielding frente a la API del SGED (Bloom 4 — Análisis)	42
A.4	Comparación con un sistema de producción y calidad ISO/IEC 25010 (Bloom 6)	42
B	Archivo referencias.bib	43

1 Introducción

El ecosistema del desarrollo web ha evolucionado aceleradamente durante la última década. La adopción masiva de arquitecturas orientadas a servicios, el auge de las API REST y la irrupción de paradigmas como Jamstack y la IA generativa obligan al profesional contemporáneo a comprender tanto los fundamentos teóricos como sus implicaciones prácticas.

Esta práctica tiene como propósito consolidar los conocimientos de la Unidad IV a través de la implementación, documentación y análisis crítico de un sistema real: el SGED (*Sports Student Management System*), desarrollado como Proyecto de Fin de Carrera (PFC) del equipo.

El informe se organiza como sigue: la Sección 2 presenta la revisión bibliográfica con al menos cinco fuentes académicas en norma APA; la Sección 4 describe la aplicación MVC completa; la Sección 5 documenta la API REST con Swagger UI; la Sección 6 elabora la comparativa SOAP vs. REST; la Sección 8 reflexiona sobre las tendencias emergentes; y las Secciones 10 y 11 presentan las conclusiones y el trabajo futuro, respectivamente.

2 Revisión Bibliográfica

Como parte de la Directriz 1, esta sección revisa seis fuentes académicas y técnicas primarias — superando el mínimo de cinco exigido — y las contrasta directamente contra el código y el comportamiento observado del SGED durante la verificación en vivo documentada en la Sección 4. La disertación de Fielding [1] y la documentación oficial de SpringDoc Team [2] se incluyen por ser, respectivamente, la fuente seminal de REST y la librería que el propio backend usa para exponer su documentación.

2.1 Fielding (2000) — REST como estilo arquitectónico

Fielding [1] define REST no como un protocolo sino como un conjunto de seis restricciones arquitectónicas — cliente-servidor, *stateless*, cacheable, interfaz uniforme, sistema por capas y código bajo demanda (opcional) — cuya combinación produce sistemas escalables y evolucionables sin acoplar cliente y servidor a una implementación concreta [1]. Verificando el SGED contra estas seis restricciones: la autenticación es efectivamente

stateless (el JWT viaja en una *cookie* `HttpOnly`, no en sesión de servidor); la interfaz es mayormente uniforme (`/api/categorias`, `/api/entrenadores`, sustantivos en plural); pero la restricción de *cacheable* no se cumple de forma consistente — no se observaron cabeceras `Cache-Control` en las respuestas inspeccionadas por Swagger durante la verificación en vivo — lo cual es una brecha real entre la teoría de Fielding y la implementación, no solo una observación de manual.

2.2 Richardson y Ruby (2007) — REST Práctico

Richardson y Ruby [3] amplían a Fielding con una guía operativa de diseño de APIs RESTful, introduciendo la *Resource-Oriented Architecture* (ROA) y un modelo de madurez por niveles (Richardson Maturity Model). Bajo ese modelo, el SGED alcanza el Nivel 2 (recursos + verbos HTTP correctos) pero no el Nivel 3 (HATEOAS): las respuestas de `GET /api/categorias/{id}` devuelven el recurso sin enlaces a acciones relacionadas (por ejemplo, a las sesiones de esa categoría), algo que sí exigiría un API verdaderamente madura según Richardson y Ruby [3].

2.3 SpringDoc OpenAPI 2 — Documentación oficial

SpringDoc Team [2] documenta la integración de SpringDoc OpenAPI 2 con Spring Boot 3, la librería que el SGED usa para exponer su especificación OpenAPI en `/api/docs.json` y la interfaz interactiva Swagger UI en `/api/docs`. Esta configuración no es la ruta por defecto de la librería (que sería `/swagger-ui/index.html`), sino un ajuste explícito del equipo:

Listing 1: Configuración SpringDoc en `application.yml`

```
springdoc:
  api-docs:
    path: /api/docs.json
  swagger-ui:
    path: /api/docs
    operationsSorter: method
```

Verificado en vivo: acceder a la ruta por defecto de SpringDoc produce el error “*Failed to load remote configuration*”, porque la especificación OpenAPI efectivamente vive en `/api/docs.json`, no en la ruta que SpringDoc asume por convención. Es un

detalle de configuración correcto pero no evidente para quien no conoce de antemano el ajuste — vale la pena documentarlo explícitamente en el **README** del proyecto, algo que actualmente no ocurre.

2.4 W3C (2007) — Especificación SOAP 1.2

W3C [4] normaliza el protocolo SOAP 1.2: mensajería basada en XML con un *Envelope* obligatorio (*Header* + *Body*), independiente del transporte subyacente, y descrita formalmente mediante WSDL. Esta fuente sustenta la comparativa SOAP vs. REST desarrollada por Pallo en la Sección 6; se cita aquí porque el contraste entre el tipado fuerte y *machine-readable* de WSDL frente al contrato implícito (y, como se documenta en la Sección 4, a veces inconsistente) de la API REST del SGED es un argumento recurrente en el análisis crítico de esta unidad.

2.5 Masse (2011) — REST API Design Rulebook

Masse [5] sistematiza reglas prescriptivas de diseño REST: nomenclatura de URIs en minúsculas con sustantivos en plural, uso semántico de verbos HTTP, y — el punto más relevante para esta revisión — gestión de errores mediante códigos de estado **consistentes y significativos**. El SGED cumple la convención de nomenclatura (`/api/categorias`, `/api/entrenadores`), pero la verificación en vivo (Sección 4) encontró una violación directa de este principio de Masse: un parámetro de *sort* inválido en `GET /api/categorias` no produce un `400 Bad Request` (el código correcto para una entrada mal formada del cliente) sino un `500 Internal Server Error`, que según Masse [5] debe reservarse para fallos genuinos del servidor, no para validación de entrada ausente.

2.6 Garriga et al. (2021) — Adopción de APIs en América Latina

Garriga et al. [6] documentan que, pese a la migración mayoritaria de la industria hacia REST, servicios SOAP legados persisten en banca y sector público de la región por compatibilidad histórica con sistemas ya certificados. Esto contextualiza por qué el ejercicio comparativo de la Sección 6 sigue siendo relevante en 2026 y no un ejercicio puramente académico: un desarrollador ecuatoriano tiene probabilidad real de integrar un servicio SOAP del sector público durante su carrera profesional.

2.7 Fielding y Reschke (2014) — RFC 7231: Semántica HTTP/1.1

Esta séptima fuente — que excede el mínimo de cinco exigido por la Directriz 1 — se incorporó específicamente a raíz del Hallazgo 2 de la verificación en vivo (Sección 4), no como bibliografía genérica de REST. Fielding y Reschke [7] define la semántica normativa de los códigos de estado HTTP: la §6.5.1 especifica que **400 Bad Request** indica que “the server cannot or will not process the request due to something that is perceived to be a client error”, mientras que la §6.6.1 reserva **500 Internal Server Error** para condiciones inesperadas que impiden al servidor completar una solicitud *por lo demás válida*. Contrastado contra este estándar, el comportamiento observado en `GET /api/categorias` — responder 500 ante un parámetro `sort` sintácticamente inválido, que es información perfectamente conocida en el momento de recibir la petición — es una clasificación incorrecta del error según la propia especificación del protocolo, no solo una preferencia de estilo de Masse (Sección 2, subsección anterior). Ambas fuentes se refuerzan mutuamente: Masse prescribe la buena práctica; el RFC 7231 es la norma que esa práctica busca cumplir.

2.8 WCAG 2.2 (W3C, 2023) — Pautas de Accesibilidad para el Contenido Web

Esta fuente respalda dos hallazgos a la vez: el 1 y el 4 (Sección 4). WCAG 2.2 [8] es el estándar vigente del W3C (recomendación de octubre de 2023) para accesibilidad web, y dos de sus criterios de éxito aplican directamente al formulario de alta de usuario del SGED. El Criterio 3.3.1 (*Identificación de Errores*) exige que, cuando se detecta un error de entrada, el elemento problemático se identifique y el error se describa al usuario en texto; el Criterio 3.3.3 (*Sugerencia ante Errores*) exige que, si se conoce, se sugiera la corrección. El mensaje fijo de `mensajeDeError(status)` — que culpa a cédula/fecha sin importar cuál campo falló — incumple ambos: no identifica el campo real ni sugiere la corrección correcta (Hallazgo 1). El Criterio 3.3.2 (*Etiquetas o Instrucciones*) exige que, cuando la entrada de usuario requiere un formato específico, ese formato se comunique en la etiqueta o instrucciones asociadas; la etiqueta “Usuario (correo de acceso)” no comunica que el valor debe tener formato de correo electrónico, lo que también incumple este criterio (Hallazgo 4). Se prefiere WCAG 2.2 sobre una fuente puramente de UX porque es una norma técnica con criterios de éxito verificables uno a uno contra el

código, no una guía de estilo general.

2.9 ISO/IEC 25010:2023 — Modelo de calidad de producto de software

Esta fuente respalda el Hallazgo 3 (Sección 4). International Organization for Standardization [9] define *completitud funcional* como una subcaracterística de la calidad funcional: el grado en que el conjunto de funciones cubre todas las tareas y objetivos del usuario especificados. El backend del SGED implementa el CRUD completo de Categorías (GET, POST, PUT, DELETE), pero el frontend Angular únicamente consume GET /api/categorias/activas para un *dropdown*; no existe ninguna pantalla desde la cual un administrador pueda crear, editar o eliminar una categoría. Bajo International Organization for Standardization [9], esto no es un defecto menor de interfaz: es un incumplimiento directo de la completitud funcional, porque una función especificada e implementada en el backend queda inaccesible para el usuario final, que es exactamente el sujeto que la norma protege.

2.10 Evans (2003) y Kleppmann (2017) — Consistencia de datos relacionados

Estas dos fuentes, una fundacional y otra contemporánea, respaldan juntas el Hallazgo 5 (Sección 4), el de mayor severidad de la revisión. Evans [10] introduce el concepto de *aggregate* (agregado): un grupo de objetos de dominio que debe tratarse como una unidad consistente, con invariantes garantizadas como parte de una sola operación, no repartidas en pasos independientes que el sistema no obliga a completar. Casi tres décadas después, Kleppmann [11] — una de las referencias más citadas en ingeniería de software de la última década sobre sistemas de datos — formaliza el mismo problema en términos operacionales: cuando dos escrituras relacionadas no ocurren dentro de la misma frontera transaccional, el sistema puede quedar en un estado parcialmente aplicado que viola una invariante de negocio, y ningún mecanismo detecta ni previene esa inconsistencia. El alta de un **Entrenador** en el SGED es exactamente ese escenario: crear la cuenta de usuario (`seguridad.usuarios`) y crear el registro profesional vinculado (`deportivo.entrenadores`, con una relación `@OneToOne` obligatoria) son dos operaciones separadas, en dos controladores distintos, sin ninguna transacción ni flujo de interfaz que las una — el resultado observado en la verificación en vivo fue precisamente

el estado intermedio inválido que ambas fuentes predicen.

3 El Patrón MVC y su Implementación en Frameworks Modernos

3.1 Historia y evolución del MVC

El patrón Modelo–Vista–Controlador nace en 1979, cuando Reenskaug [12] lo describe trabajando sobre Smalltalk-80 en Xerox PARC. Su formulación original responde a un problema de interfaces gráficas de escritorio: el *modelo* representa el estado y el conocimiento del dominio, la *vista* es una representación visual de ese modelo, y el *controlador* es el intermediario que traduce los gestos del usuario —teclado y ratón— en operaciones sobre el modelo. La separación perseguía que un mismo modelo pudiera tener varias vistas simultáneas y que cambiar la presentación no obligara a tocar la lógica.

La web adaptó el patrón cambiándole tres piezas. El controlador ya no atiende eventos de teclado sino **solicitudes HTTP** sin estado, que llegan aisladas unas de otras. La vista se renderiza en el servidor mediante un motor de plantillas, o bien —como en el SGED— se traslada por completo al cliente en forma de *Single Page Application*, y el servidor pasa a devolver JSON en lugar de HTML. Y el modelo deja de ser una clase única para convertirse en una capa: entidades de dominio, repositorios que las persisten y servicios que encapsulan las reglas de negocio.

De esa adaptación derivan dos variantes de uso corriente. **MVP** (Model–View–Presenter) elimina la referencia directa entre vista y modelo: todo pasa por el presentador, lo que facilita las pruebas al hacer la vista completamente pasiva. **MVVM** (Model–View–ViewModel) introduce enlace de datos bidireccional entre la vista y un modelo de vista, y es el esquema que siguen de facto los frameworks de SPA. Conviene señalar aquí una consecuencia honesta para este proyecto: el SGED es MVC en el backend y MVVM en el frontend Angular, de modo que hablar de “la arquitectura MVC del sistema” sin ese matiz sería impreciso.

3.2 El patrón Front Controller

Todos los frameworks MVC web comparten un punto de entrada único —`index.php` en Laravel, `Program.cs` en ASP.NET Core, el `DispatcherServlet` en Spring MVC— que Fowler [13] formaliza como *Front Controller*. La razón es que cuatro preocupaciones son transversales a todas las peticiones y no deben duplicarse en cada manejador: el enrutamiento, la autenticación, el registro de actividad y el manejo de errores. Centralizarlas en un solo punto convierte esas responsabilidades en configuración en lugar de código repetido.

El ciclo completo de una petición en el SGED, tomando `GET /api/categorias/activas` como caso real, recorre seis pasos:

1. El `DispatcherServlet` de Spring MVC recibe la petición: es el Front Controller del framework y el único punto de entrada del backend.
2. La cadena de filtros de Spring Security la intercepta antes de cualquier controlador. `JwtAuthenticationFilter` —registrado con `addFilterBefore(..., UsernamePasswordAuthenticationFilter.class)` en `SecurityConfig`— lee el JWT de la cookie *HttpOnly*, valida su firma y puebla el `SecurityContext`.
3. El `HandlerMapping` resuelve la ruta al método `CategoriaController.listarActivas()` a partir de las anotaciones `@RequestMapping` y `@GetMapping`.
4. El interceptor de seguridad de método evalúa `@PreAuthorize("hasAnyRole(...)")` contra los roles del contexto y corta con 403 si no corresponde.
5. El controlador delega en `CategoriaService`, que consulta la caché en Redis y, ante un fallo de caché, va a `CategoriaRepository` (Spring Data JPA) y de ahí a PostgreSQL.
6. El valor devuelto pasa por un `HttpMessageConverter`, que con Jackson serializa el *record* `CategoriaResponse` a JSON y lo escribe en la respuesta.

Obsérvese que los pasos 2 y 4 son exactamente las preocupaciones transversales que justifican el patrón: ningún controlador del SGED contiene código de autenticación.

Cuadro 1: Comparativa de frameworks MVC web vigentes

Criterio	Laravel 11	Symfony 7	Spring Boot 3	ASP.NET Core 8	Django 5
Lenguaje	PHP 8.2+	PHP 8.2+	Java 17/21	C# 12	Python 3.10+
Paradigma	Convención sobre configuración	Explícito, muy configurable	Autoconfiguración con anulación explícita	Convención con inyección explícita	Convención (<i>batteries included</i>)
ORM incluido	Eloquent (Active Record)	Doctrine (Data Mapper)	Spring Data JPA / Hibernate	Entity Framework Core	Django ORM
Motor de plantillas	Blade	Twig	Thymeleaf	Razor	Django Templates
Ecosistema	Muy amplio, orientado a productividad	Amplio, componentes reutilizables	Enterprise, muy extenso	Integrado con el ecosistema Microsoft	Amplio en ciencia de datos
Rendimiento (Round 22)	Franja baja	Franja baja-media	Franja media	Franja alta	Franja baja

3.3 Comparativa de frameworks MVC

La fila de rendimiento refleja la posición relativa reportada en la Ronda 22 de los *TechEmpower Framework Benchmarks*, publicada el 15 de noviembre de 2023 [14]: ASP.NET Core encabeza el grupo de frameworks de propósito general, mientras que Express, Rails, Laravel y Django quedan agrupados muy cerca entre sí en la parte baja de la tabla, y Spring se sitúa en una franja intermedia. Se consigna la franja y no una cifra puntual de *composite score* porque esos valores dependen del tipo de prueba (JSON, consultas únicas, múltiples, *Fortunes*, actualizaciones y texto plano) y del hardware del entorno Citrine; citar un único número descontextualizado daría una precisión aparente que la fuente no respalda. Corresponde además una advertencia metodológica: estos benchmarks miden *frameworks* bajo carga sintética, no aplicaciones reales, donde el cuello de botella suele estar en el diseño de las consultas y no en el framework.

3.4 Justificación del framework del PFC

La elección quedó registrada en el ADR-001 del proyecto, escrito en PE-U3, y conviene ser exacto sobre cómo se tomó: el espacio de decisión estaba **acotado por el docente**

a PHP 8.2, ASP.NET Core 8 y Java, de modo que Symfony y Django nunca fueron opciones reales aunque aparezcan en la tabla comparativa. Dentro de ese margen se eligió **Java 21 con Spring Boot 3.2.5** por cuatro razones consignadas en el ADR: Spring Data JPA resuelve el acceso a PostgreSQL —motor ya fijado como restricción—, Spring Security aporta autenticación JWT con roles de forma integrada, Angular consume un backend Java por HTTP/JSON sin capas intermedias, y el tipado estático de Java reduce errores en tiempo de ejecución en un equipo de tres personas que trabaja en paralelo sobre el mismo código.

El propio ADR documenta el costo de esa decisión, que es la parte que suele omitirse: una curva de aprendizaje estimada en dos semanas sobre diez de desarrollo efectivo, y un hospedaje más caro que el de PHP compartido. Contrastado con la tabla anterior, el resultado es coherente: si el criterio hubiera sido exclusivamente el rendimiento, ASP.NET Core habría sido la opción indicada; si hubiera sido la velocidad de arranque del equipo, Laravel. Se privilegió en cambio la integración con PostgreSQL y la robustez del modelo de seguridad, que para un sistema que maneja datos de menores de edad pesa más que unos milisegundos por petición.

4 Aplicación MVC Completa — SGED

4.1 Descripción general

El SGED es un sistema web para la gestión de una escuela deportiva, que cubre autenticación y roles, catálogo de categorías, registro de estudiantes y representantes, sesiones de entrenamiento y control de asistencia mediante códigos QR. Esta sección documenta la verificación en vivo realizada personalmente sobre el sistema levantado con Docker Compose en un entorno limpio, incluyendo el análisis crítico exigido por la actividad de revisión entre pares: qué funciona correctamente, qué no, y por qué.

4.2 Arquitectura MVC

- **Modelo + Controlador (Backend):** Spring Boot 3.2 / Java 21. Las entidades JPA se mapean a esquemas PostgreSQL 16 separados por dominio (seguridad, deportivo, academico). Los controladores REST exponen los recursos con `@RestController + @RequestMapping("/api/...")`, con autorización

declarativa por rol vía `@PreAuthorize`. La capa de servicio (`@Service`) encapsula la lógica de negocio.

- **Vista (Frontend):** Angular (*standalone components*). La comunicación con el backend usa el `HttpClient` de Angular con `withCredentials: true` para el manejo automático de la *cookie* JWT.

4.3 Módulos verificados

Cuadro 2: Módulos del SGED y resultado de la verificación en vivo

Módulo	Backend	Resultado verificado
Autenticación	<code>AuthController</code> + <code>JWT cookie</code>	Correcto: login, sesión y <i>logout</i> funcionan de punta a punta.
Registro de estudiante	<code>EstudianteController</code>	Correcto: alta completa sin errores.
Alta de usuario	<code>AuthController.register</code>	Funcional, pero con un defecto de UX (Hallazgo 1).
Categorías (backend)	<code>CategoriaController</code> CRUD	CRUD completo y correcto vía API; con un defecto de robustez (Hallazgo 2).
Categorías (frontend)	—	Sin interfaz de administración (Hallazgo 3).
Sesiones	<code>SesionEntrenamientoController</code>	Controlado para ENTRENADOR hasta resolver un defecto de flujo (Hallazgo 5); funcional tras el fix.

4.4 Verificación en laboratorio

El sistema se levantó con `docker compose up -d --build` sobre una copia limpia del repositorio, confirmando los cuatro contenedores (`sged_postgres`, `sged_redis`, `sged_backend`, `sged_frontend`) en estado *healthy*. A partir de ahí se ejecutó el flujo completo: login como `admin`, registro de un estudiante real, alta de un usuario con rol `ENTRENADOR`, y navegación por Swagger UI para inspeccionar cada recurso expuesto por la API.

Esta verificación no se limitó a confirmar que el sistema *corre*, sino a documentar con evidencia reproducible dónde el comportamiento observado se aparta del esperado. Se encontraron cinco hallazgos, ordenados de menor a mayor severidad:

1. **Mensaje de error genérico e impreciso (severidad media, UX).** El formulario de alta de usuario traduce *cualquier* respuesta 422 del backend al mismo texto fijo (“*la cédula debe tener 10 dígitos y la fecha de nacimiento ser pasada*”), sin importar cuál campo violó realmente la validación. En la verificación, una cédula y fecha de nacimiento válidas fueron señaladas como erróneas cuando el campo que en realidad fallaba — por formato de correo inválido en “Usuario (correo de acceso)” — no se mencionaba en absoluto. Código relevante: `frontend/src/app/features/admin/crear-usuario.component.ts`, método `mensajeDeError(status)`. Es una violación medible del Criterio de Éxito 3.3.1 (*Identificación de Errores*) de World Wide Web Consortium (W3C) [8] (Sección 2): el mensaje no identifica el campo real que causó el error.
2. **500 en vez de 400 ante un parámetro inválido (severidad media, robustez).** `GET /api/categorias` vincula el parámetro de consulta `sort` directamente a un `Pageable` de Spring Data sin validarlo; un valor de `sort` que no corresponde a una columna real de la entidad produce una excepción SQL no controlada que el manejador global traduce en un `500 Internal Server Error`. Esto no es solo una preferencia de estilo: según Masse [5] (Sección 2) y la semántica normativa de Fielding y Reschke [7], `5xx` está reservado a fallos genuinos del servidor (§6.6.1 de Fielding y Reschke [7]), mientras que una entrada del cliente que no puede procesarse por estar mal formada corresponde a `400 Bad Request` (§6.5.1 de Fielding y Reschke [7]). Devolver 500 aquí no solo es una respuesta técnicamente incorrecta: oculta al cliente que el error es corregible ajustando su propia petición, y en un entorno de producción dispararía falsas alarmas de monitoreo de infraestructura por un problema que en realidad es de validación de entrada. Se verificó y reprodujo con un test de integración propio contra base de datos real (ver Sección 4).
3. **CRUD de Categorías sin interfaz (severidad media, funcionalidad incompleta).** El backend expone `GET`, `POST`, `PUT` y `DELETE` completos para `/api/categorias`, pero el frontend Angular únicamente consume `GET /api/categorias/activos` para llenar un *dropdown* en el formulario de registro de estudiante. No existe ninguna pantalla desde la cual un administrador pueda crear, editar o dar de baja una categoría sin usar la API directamente. Bajo el modelo de calidad de

International Organization for Standardization [9] (Sección 2), esto es un incumplimiento directo de la subcaracterística de completitud funcional: una función implementada y correcta en el backend queda inaccesible para el usuario final.

4. **Etiqueta de campo ambigua (severidad baja, UX).** El campo llamado “Usuario (correo de acceso)” es, en el backend, un correo electrónico obligatorio (@Email en RegisterRequest), pero la etiqueta no lo deja claro y el campo no tiene validación de formato en el cliente antes de enviar el formulario — el usuario descubre el requisito solo al recibir el mensaje genérico del Hallazgo 1. Incumple el Criterio de Éxito 3.3.2 (*Etiquetas o Instrucciones*) de World Wide Web Consortium (W3C) [8] (Sección 2), que exige comunicar el formato esperado cuando la entrada requiere uno específico.
5. **Alta de Entrenador incompleta por diseño (severidad alta, funcionalidad crítica).** Crear una cuenta con rol ENTRENADOR desde el panel de administración solo inserta una fila en seguridad.usuarios; el sistema mantiene una tabla separada, deportivo.entrenadores, con una relación @OneToOne obligatoria hacia ese usuario (campo profesional: especialidad, años de experiencia, certificación). El rol se otorga y permite el acceso al módulo de Sesiones, pero la lógica de negocio — que busca el registro de Entrenador vinculado, no solo el rol — rechaza toda acción con el mensaje “No hay un entrenador asociado a esta cuenta”. El endpoint que crea ese vínculo (POST /api/entrenadores) existe y funciona correctamente, pero está restringido a ADMINISTRADOR y no tiene ninguna pantalla en el frontend: la única forma de completarlo es invocando la API directamente. Se reprodujo el defecto completo (alta de usuario ENTRENADOR → bloqueo confirmado en “Mis sesiones” → vínculo creado vía Swagger con POST /api/entrenadores → módulo de Sesiones verificado funcional) para aislar la causa raíz y confirmar la solución, no solo reportar el síntoma. En términos de Evans [10] y, más recientemente, Kleppmann [11] (ambos en Sección 2), el “Entrenador” debería tratarse como un agregado cuya creación es una sola operación consistente, dentro de una frontera transaccional única; el SGED lo reparte en dos escrituras independientes sin garantizar la invariante entre ellas, lo que permite que el sistema quede en un estado intermedio inválido sin advertirlo.

Fuera de estos cinco hallazgos, el flujo de autenticación, el registro de estudiantes y la documentación Swagger UI (una vez corregida la ruta de acceso, ver Hallazgo relacionado en la Sección 2, subsección SpringDoc) funcionaron correctamente en cada intento.

La tabla completa de hallazgos — con causa raíz, severidad y evidencia trazable, en el mismo formato de seguimiento que el propio equipo SGED usa en `docs/observaciones/OBSERVACIONES.md` — queda documentada en `docs/observaciones/REVISION-BELTRAN.md`. El Hallazgo 2 (500 en vez de 400 ante un `sort` inválido) se reprodujo además con un test de integración propio contra base de datos real, no mocks: `backend/src/test/java/org/uteq/backend/deportivo/categoria/CategoriaSortValidationIntegrationTest.java`, verificado en verde localmente (`mvnw test -Dtest=CategoriaSortValidationIntegrationTest`).

5 Diseño de la API REST: Principios de Fielding, JWT y OpenAPI

5.1 Los seis principios de Fielding en el diseño de la API

Fielding [1] no define REST como un formato sino como un conjunto de seis restricciones arquitectónicas. La API del SGED las adopta de forma desigual, y esa desigualdad está analizada una por una —con el costo concreto de cada incumplimiento— en el Anexo A. En síntesis: *cliente-servidor* y *sistema por capas* se cumplen por completo; *sin estado* e *interfaz uniforme* de forma parcial; *cacheable* no se cumple, porque Spring Security emite `Cache-Control: no-store` en los endpoints autenticados; y *código bajo demanda* no aplica por ser opcional. Los dos apartados siguientes desarrollan las dos restricciones sobre las que el equipo tomó decisiones explícitas de diseño: la interfaz uniforme, a través de las URIs, y la ausencia de estado, a través del JWT.

5.2 Diseño de URIs: reglas aplicadas y anti-patrones detectados

La regla central es que una URI identifica un **recurso** —un sustantivo— y que la operación la expresa el verbo HTTP, no la ruta [5]. La API sigue esa regla en su núcleo: `GET /api/estudiantes` lista, `POST /api/estudiantes` crea, `PUT /api/estudiantes/{id}` actualiza y `DELETE /api/estudiantes/{id}` da de baja. La jerarquía se expresa por anidamiento cuando el recurso hijo solo existe en el contexto del padre, como en `GET`

/api/lesiones/estudiante/{idEstudiante} o POST /api/representantes/{id}/estudiantes/

Ahora bien, una revisión honesta del mapa completo de rutas —dieciséis controladores— revela cuatro desviaciones que conviene declarar antes de que las señale el evaluador.

Cuadro 3: Anti-patrones detectados en las URIs del SGED

Anti-patrón	Casos en la API	Costo real
Verbo en la ruta	/api/estudiantes/operaciones/ idEstudiante , no /api/consentimientos/{id}/revocar, no /api/lesiones/{id}/alta, /api/asistencias/qr/marcar	Son desactivaciones de estado, no recursos. Un diseño estricto las modelaría como PATCH sobre el recurso o como un sub-recurso de estado. El costo es de previsibilidad: el cliente no puede deducir la ruta, tiene que consultarla.
Singular frente a plural	/api/representante (portal del representante autenticado) convive con /api/representantes (CRUD administrativo)	La distinción es real —uno es autoservicio, el otro gestión— pero la diferencia de una sola letra la vuelve indistinguible de una errata. Un nombre como /api/representantes/me habría expresado lo mismo sin ambigüedad.
Convención mixta	/api/estados_generales usa <i>snake_case</i> ; el resto de la API usa una sola palabra en minúsculas	Rompe la uniformidad y obliga a recordar la excepción.
Ausencia de versionado	Ninguna ruta lleva /v1/, ni se negocia versión por cabecera o parámetro	Hoy el costo es bajo porque el frontend se despliega junto al backend. Deja de serlo en cuanto exista un consumidor externo: un cambio incompatible rompería sin aviso a la colección Postman versionada y a cualquier integración.

De las tres estrategias de versionado que suelen compararse —por ruta (/api/v1/), por cabecera (Accept-Version) y por parámetro de consulta—, la primera sería la adecuada aquí: es la más visible en registros y en la propia colección Postman, y no exige que el cliente recuerde enviar nada adicional. Se registra como trabajo pendiente y no como decisión tomada.

5.3 JWT: estructura, flujo y revocación

Un JSON Web Token tiene tres partes separadas por puntos y codificadas en Base64URL. La **cabecera** declara el algoritmo de firma; el **payload** transporta las afirmaciones (*claims*); la **firma** es el HMAC de las dos anteriores con la clave del servidor, y es lo que impide alterar el contenido. El SGED firma con **HS256** (HMAC-SHA-256), que es lo que la librería selecciona a partir de una clave simétrica de al menos 256 bits.

Listing 2: Payload real emitido por JwtService (valores de ejemplo)

```
{
  "sub": "admin",           // usuario autenticado
  "rol": "ADMINISTRADOR",   // claim propio: autoridad para
                             @PreAuthorize
  "type": "access",         // access | refresh: impide
                             usar uno por el otro
  "iss": "sged-backend",    // emisor, validado con
                             requireIssuer
  "aud": "sged-frontend",   // audiencia, validada en
                             isTokenValid
  "jti": "8f3c1e0a-...",    // identificador unico: la
                             clave de la revocacion
  "iat": 1786512000,
  "exp": 1786515600         // 1 h para el access; 7 dias
                             para el refresh
}
```

El flujo completo se aparta del esquema de manual en un punto deliberado. El manual dice: el cliente guarda el token y lo envía en la cabecera **Authorization: Bearer**. El SGED **no hace eso**. Tras validar credenciales, el servidor emite el token dentro de una cookie `sged_access` marcada `HttpOnly + Secure + SameSite=Strict`; el navegador la reenvía sola en cada petición y `JwtAuthenticationFilter` la lee del lado del servidor, verifica firma, emisor, audiencia y expiración, y puebla el contexto de seguridad. El frontend Angular nunca ve el token ni lo adjunta a mano. El motivo, registrado en los ADR-002 y ADR-007, es que un token legible por JavaScript convierte cualquier XSS en una toma de sesión inmediata.

Queda entonces el problema que todo esquema *stateless* arrastra: **un JWT firmado es válido hasta que expira**, y el servidor no tiene dónde “borrarlo”. Las dos estrategias habituales son acortar mucho la vida del token y aceptar la ventana de riesgo, o mantener una lista de revocación. El SGED implementa la segunda con la clase `RedisBlacklistService`: al cerrar sesión, el `jti` del token se almacena en Redis con un TTL igual al tiempo que le restaba de vida, y el filtro rechaza cualquier token cuyo `jti` figure ahí. La entrada se autoelimina justo cuando el token habría caducado por sí solo, de modo que la lista nunca crece indefinidamente.

Conviene ser explícito sobre lo que esto cuesta, porque es la contradicción que el Anexo C señala: consultar una lista de revocación es, por definición, consultar estado del servidor, así que la API no es estrictamente *stateless*. Se aceptó a cambio de que cerrar sesión signifique algo. La alternativa —no revocar— habría preservado la pureza del principio y dejado sesiones robadas vivas hasta una hora.

5.4 OpenAPI 3 como contrato: Swagger UI y Redoc

La especificación OpenAPI cumple entre frontend y backend la misma función que el WSDL cumple en SOAP: describir el contrato en un formato legible por máquinas, de modo que un consumidor pueda generar un cliente sin leer el código del servidor. En el SGED no se escribe a mano: `springdoc` la deriva de las anotaciones de Spring MVC, de las firmas de los métodos y de los *records* de DTO, y la publica en `/api/docs.json`. Que el contrato se genere del código y no de un documento paralelo evita la divergencia habitual entre lo que la API promete y lo que hace.

Cada operación queda descrita por su `operationId`, sus `parameters` —con nombre, ubicación y tipo—, su `requestBody` y el conjunto de `responses` con el esquema de cada código de estado. Para el endpoint de clima incorporado en la Sección 7, por ejemplo, la especificación generada expone la ruta `/api/clima/entrenamiento` con sus tres parámetros opcionales (`fecha`, `desde` y `hasta`) sin que se haya escrito una sola anotación de documentación.

Sobre esa misma especificación pueden montarse dos interfaces de propósito distinto. **Swagger UI** es interactiva: permite ejecutar peticiones desde el navegador, lo que la hace la herramienta adecuada para desarrollo y para una demostración en vivo como la de esta práctica, y es la que el SGED sirve en `/api/docs`. **Redoc** genera documentación estática

de tres columnas, más legible y mejor indexable, pero sin capacidad de ejecución: es la opción razonable para publicar documentación a consumidores externos, precisamente porque no ofrece un botón que dispare llamadas reales contra el servidor.

5.5 Configuración de Swagger UI

La API del SGED está documentada mediante **SpringDoc OpenAPI 2**. Para cumplir la Directriz 3, se ajustó `application.yml` de modo que la interfaz interactiva Swagger UI quede accesible en `/api/docs`:

Listing 3: Rutas SpringDoc ajustadas para la demo

```
springdoc:
  api-docs:
    path: /api/docs.json      # JSON crudo OpenAPI 3.1
  swagger-ui:
    path: /api/docs          # Interfaz interactiva Swagger UI
    operationsSorter: method
```

5.6 Endpoints demostrados en vivo

Los tres endpoints se ejecutan en el siguiente orden durante la sesión de laboratorio.

5.6.1 Endpoint 1 — POST `/api/auth/login`

Listing 4: Request body — POST `/api/auth/login`

```
{
  "username": "admin",
  "password": "Admin2026!"
}
```

Respuesta esperada (200 OK):

Listing 5: Response — POST `/api/auth/login`

```
{
  "username": "admin",
  "role": "ADMINISTRADOR",
}
```

```
"message": "Sesion iniciada correctamente"
}
```

La cookie `access_token` se incluye automáticamente en todas las peticiones siguientes si el manejo de cookies está activo en Postman/Insomnia.

5.6.2 Endpoint 2 — GET `/api/categorias/activas`

Retorna la lista de categorías activas; requiere rol ADMINISTRADOR, ENTRENADOR o RECEPCIONISTA.

Listing 6: Response — GET `/api/categorias/activas` (captura real, base de datos del SGED)

```
[
  {
    "idCategoria":1,"nombre":"SUB-12","edadMin":10,"edadMax":12,
    "descripcion":"Categoria sub-12","activo":true,
    "createdAt":"2026-08-03T04:44:33.481682Z"},
  {
    "idCategoria":2,"nombre":"SUB-14","edadMin":12,"edadMax":14,
    "descripcion":"Categoria sub-14","activo":true,
    "createdAt":"2026-08-03T04:44:33.481682Z"},
  {
    "idCategoria":3,"nombre":"SUB-16","edadMin":14,"edadMax":16,
    "descripcion":"Categoria sub-16","activo":true,
    "createdAt":"2026-08-03T04:44:33.481682Z"}
]
```

Esta salida no es un ejemplo ilustrativo: es la respuesta real capturada contra la base de datos del SGED, con las tres categorías sembradas el 3 de agosto de 2026. La ejecución en Postman está documentada en [docs/postman/evidencias/](#).

5.6.3 Endpoint 3 — POST `/api/asistencias/qr/sesion/{idSesion}/token`

Genera el token QR para una sesión de entrenamiento. Requiere rol ADMINISTRADOR o RECEPCIONISTA y un `idSesion` válido previamente creado.

Listing 7: Response — POST `/api/asistencias/qr/sesion/1/token`

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
}
```

```
"expiresAt": "2026-08-09T02:30:00Z",  
"idSesion": 1  
}
```

Plan B: si no hay sesión preexistente, sustituir por `POST /api/categorias` (misma categoría de “endpoint que escribe”, sin dependencias de datos previos).

5.7 Checklist de ensayo previo al laboratorio

Checklist — Demo API REST en vivo

- ☐ Rutas SpringDoc ajustadas en `application.yml`.
- ☐ `make up` ejecutado; stack completo en pie.
- ☐ `http://localhost:8080/api/docs` abre Swagger UI (no JSON).
- ☐ Colección Postman/Insomnia guardada; cookies activas.
- ☐ Flujo completo ensayado: login → categorías activas → token QR.
- ☐ Plan B preparado (`POST /api/categorias`).

6 Comparativa SOAP vs. REST

6.1 Tabla de criterios

6.2 Ejemplo SOAP y su equivalente REST

6.2.1 SOAP — NumberToWords (servicio público verificado con curl)

Listing 8: Request SOAP — POST NumberToWords

```
POST https://www.dataaccess.com/webservicesserver/  
    NumberConversion.wso  
Content-Type: text/xml; charset=utf-8  
SOAPAction: http://www.dataaccess.com/webservicesserver/  
    NumberToWords  
  
<?xml version="1.0" encoding="utf-8"?>  
<soap:Envelope
```

Cuadro 4: Comparativa SOAP vs. REST — 9 criterios de análisis

Criterio	SOAP	REST
Formato del mensaje	XML estricto, dentro de un <i>Envelope</i> (Header + Body)	Flexible; en la práctica casi siempre JSON
Contrato del servicio	WSDL obligatorio	OpenAPI / Swagger opcional (SGED: <i>/api/docs</i>)
Transporte	Agnóstico en teoría; HTTP en la práctica	Exclusivamente HTTP con verbos semánticos
Overhead	Alto: <i>Envelope</i> XML + cabeceras WS-*	Bajo: JSON compacto, sin envoltorio obligatorio
Seguridad	WS-Security a nivel de mensaje	TLS + JWT / OAuth2 (SGED: JWT en <i>cookie HttpOnly</i>)
Manejo de errores	<i>SOAP Fault</i> estandarizado (<i>faultcode/faultstring</i>)	Códigos HTTP estándar + cuerpo de error propio
Consumo desde móvil	Pesado: parseo XML + librerías desde WSDL	Liviano: JSON nativo en cualquier SDK móvil
Caché	No aprovecha caché HTTP (todo son POST)	GET cacheables con cabeceras HTTP estándar
Casos de uso en Ecuador	Sistemas legados bancarios y gubernamentales [6]	Apps nuevas, <i>fintechs</i> y apps móviles (incluido SGED)

```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
<soap:Body>
  <NumberToWords
    xmlns="http://www.dataaccess.com/webservicesserver/"
    <ubiNum>500</ubiNum>
  </NumberToWords>
</soap:Body>
</soap:Envelope>

```

Listing 9: Response SOAP — captura real

```

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  <soap:Body>

```

```

    <m:NumberToWordsResponse
      xmlns:m="http://www.dataaccess.com/webservicesserver/">
      <m:NumberToWordsResult>five hundred </
        m:NumberToWordsResult>
    </m:NumberToWordsResponse>
  </soap:Body>
</soap:Envelope>

```

6.2.2 REST — Equivalente en SGED

Listing 10: Request REST — GET /api/categorias/activas

```

GET /api/categorias/activas HTTP/1.1
Host: localhost:8080
Cookie: access_token=<obtenida en POST /api/auth/login>

```

Listing 11: Response REST — array JSON directo (sin envoltorio), captura real

```

[
  {"idCategoria":1,"nombre":"SUB-12","edadMin":10,"edadMax":12,
    "descripcion":"Categoria sub-12","activo":true,
    "createdAt":"2026-08-03T04:44:33.481682Z"},
  {"idCategoria":2,"nombre":"SUB-14","edadMin":12,"edadMax":14,
    "descripcion":"Categoria sub-14","activo":true,
    "createdAt":"2026-08-03T04:44:33.481682Z"},
  {"idCategoria":3,"nombre":"SUB-16","edadMin":14,"edadMax":16,
    "descripcion":"Categoria sub-16","activo":true,
    "createdAt":"2026-08-03T04:44:33.481682Z"}
]

```

6.3 Análisis del contraste

Para devolver información útil comparable, SOAP necesita el *Envelope* XML completo (namespaces, Header/Body, nombre de la operación repetido) solo para “empacar” un resultado; REST entrega el arreglo JSON directamente. Este es el argumento empírico

del criterio *overhead*: no una afirmación teórica, sino una diferencia observable en los bytes transferidos.

Ambos lados de la comparación son ejecuciones reales y no reconstrucciones: la llamada SOAP se realizó con `curl` contra el servicio público `NumberConversion` y la respuesta está transcrita literalmente; la llamada REST se ejecutó contra la base de datos del SGED levantada en local, con evidencia en `docs/postman/evidencias/`.

6.4 Sobre el formato de respuesta de la API

Cabe justificar una decisión que se aparta de la envoltura `{success, data, message, errors, meta}` sugerida en los objetivos de la práctica. El SGED devuelve el recurso directamente y delega la señalización de éxito o error al protocolo: código 200 con el cuerpo del recurso, 201 con la entidad creada, 204 sin cuerpo, y para los errores un documento *Problem Details* conforme al RFC 7807 [15], que es lo que produce su `GlobalExceptionHandler`.

El argumento no es de comodidad sino de coherencia con el principio de interfaz uniforme de Fielding [1]: una envoltura propia duplica en el cuerpo información que HTTP ya transporta en la línea de estado, y obliga a todo cliente a leer dos veces el mismo hecho —una en el código HTTP y otra en el campo `success`— con el riesgo de que ambos se contradigan. El RFC 7807, además, es un estándar con registro IANA, mientras que la envoltura es una convención de casa. Se reconoce el costo de la decisión: los clientes que esperan una forma uniforme para toda respuesta deben inspeccionar el código de estado, y la paginación viaja en un objeto `Page` de Spring Data en vez de un campo `meta` homogéneo.

7 Consumo de Servicios Web Externos

7.1 La API externa elegida y por qué

El SGED consume **Open-Meteo** [16], un servicio REST público de pronóstico meteorológico. La elección atendió a dos criterios explícitos.

El primero es de pertinencia con el dominio. La escuela entrena en cancha abierta y la lluvia es la causa real por la que una sesión se reprograma; el dato consumido alimenta una decisión operativa concreta del entrenador, no un requisito de forma. El

segundo es de reproducibilidad: Open-Meteo no exige clave de API, de modo que el sistema levanta completo desde una clonación limpia. Este criterio pesó por experiencia propia del proyecto: el módulo de generación de texto con IA sí requiere credencial y por eso viene deshabilitado por omisión, lo que lo vuelve imposible de demostrar ante un evaluador que no tiene esa clave.

7.2 Cliente HTTP del lado del servidor

Spring Boot ofrece tres clientes: `RestTemplate`, en mantenimiento desde Spring 5; `WebClient`, reactivo y no bloqueante, pensado para WebFlux; y `RestClient`, introducido en Spring Framework 6.1, con API fluida y semántica síncrona. El SGED usa `RestClient`, coherente con su modelo de hilos bloqueante sobre Spring MVC. La configuración fija un tiempo de espera simétrico de cinco segundos en conexión y lectura, por una razón de producto antes que técnica: el clima es información de apoyo, y es preferible pintar la pantalla de sesiones sin el pronóstico que dejar al entrenador esperando frente a un proveedor lento.

7.3 Caché de respuestas externas: *patrón cache aside*

Cachear la respuesta externa persigue los tres objetivos que señala la literatura de resiliencia [17]: no agotar la cuota del proveedor, reducir la latencia percibida y aislar al sistema de la indisponibilidad del tercero. El SGED implementa *cache aside* sobre Redis con un TTL de **10 minutos**, valor acorde al dominio —el pronóstico horario no se recalcula más rápido que eso— y coincidente con el sugerido en la guía para datos meteorológicos.

La implementación es explícita y no declarativa con `@Cacheable`, a diferencia del resto del proyecto, por dos motivos que la anotación no permite resolver. El primero es que **un fallo del proveedor no debe cachearse**: con la anotación, la respuesta de degradación quedaría almacenada y el sistema seguiría informando “no disponible” durante los diez minutos del TTL aunque el proveedor se recuperara al instante siguiente. El segundo es que la respuesta declara su procedencia (`api-externa` o `cache`), dato que solo puede resolverse donde se sabe si hubo acierto de caché, y que además hace el mecanismo demostrable en vivo.

Listing 12: Cache aside en `ClimaService` — los tres pasos del patrón

```
// 1. Consultar el cache.
Optional<PronosticoEntrenamiento> enCache = leerDeCache(clave);
if (enCache.isPresent()) {
    return ClimaEntrenamientoResponse.desdeCache(enCache.get());
}
// 2. Fallo de cache: ir a la API externa.
try {
    JsonNode crudo = client.pronosticoHorario(latitud, longitud,
        zonaHoraria, fecha);
    PronosticoEntrenamiento pronostico = interpretar(crudo, fecha,
        desde, hasta);
    // 3. Guardar solo el exito.
    guardarEnCache(clave, pronostico);
    return ClimaEntrenamientoResponse.desdeApiExterna(pronostico);
} catch (RestClientException e) {
    return ClimaEntrenamientoResponse.noDisponible(
        "El servicio de clima no responde en este momento");
}
```

7.4 Manejo de errores y degradación

El endpoint responde **200 con disponible=false** y un motivo legible cuando el proveedor falla, en lugar de propagar un 5xx: la caída de un tercero no es un error del SGED, y la pantalla de sesiones debe seguir siendo utilizable sin el pronóstico. Los tiempos de espera agotados, los errores 4xx y los 5xx se capturan por igual como `RestClientException`, y un cuerpo con forma inesperada —200 con datos incompletos— recibe el mismo trato.

Redis recibe una consideración análoga: es caché, no fuente de verdad. Sus fallos se registran y el flujo continúa hacia la API externa, de modo que un Redis caído cueste velocidad y no funcionalidad. Esto responde de forma directa a una de las preguntas previstas para la defensa oral.

7.5 Verificación y métricas

La clave almacenada se verificó directamente en el contenedor de Redis:

Listing 13: Verificación del caché en Redis

Cuadro 5: Rendimiento con y sin caché — GET /api/clima/entrenamiento

Escenario	Origen	Tiempo	Salida a Internet
Primera llamada (caché vacío)	api-externa	1310 ms	Sí
Segunda llamada (dentro del TTL)	cache	32 ms	No
Efecto del caché		≈41×	1 llamada evitada

```
$ docker exec sged_redis redis-cli KEYS "clima:*"
clima:entrenamiento:2026-08-11:15:00-19:00

$ docker exec sged_redis redis-cli TTL clima:entrenamiento
:2026-08-11:15:00-19:00
598
```

La clave incorpora la franja horaria y no solo el día, porque dos sesiones del mismo día en horarios distintos tienen pronósticos distintos y no deben compartir entrada. Se forzó además un fallo real del proveedor solicitando una fecha fuera de su rango de cobertura: la respuesta degradó correctamente y, comprobado con `redis-cli`, **no dejó ninguna clave cacheada**, que es precisamente el comportamiento que motivó descartar `@Cacheable`.

El comportamiento está cubierto por seis pruebas automatizadas que no salen a Internet —el cliente está simulado—, y que verifican las decisiones propias y no el funcionamiento del proveedor: acierto y fallo de caché, no cacheo de fallos, Redis caído, respuesta incompleta y separación de claves por franja. La suite completa del backend tras el cambio reporta `Tests run: 211, Failures: 0, Errors: 0`.

7.6 Presentación en la interfaz

El pronóstico se muestra en una tarjeta “Clima en la cancha” encabezando la pantalla *Mis sesiones* del entrenador, con la temperatura máxima, la probabilidad de lluvia y la precipitación acumulada de la franja, además de una recomendación operativa. Una insignia indica si el dato provino de la API externa o del caché, lo que permite

demostrar el mecanismo sin abrir una terminal. Cuando la sesión de hoy existe, el frontend envía su franja horaria real; si no, el backend resuelve con el horario vespertino configurado por omisión.

8 Tendencias Emergentes en el Desarrollo Web

8.1 Jamstack: desacoplamiento como paradigma de arquitectura

Jamstack (*JavaScript, APIs, Markup*) es una filosofía de arquitectura web que separa radicalmente el frontend del backend: el cliente se entrega como HTML/CSS/JS pre-generado y servido desde un CDN (*Content Delivery Network*), mientras que la lógica de negocio se encapsula en APIs independientes que se consumen bajo demanda [18].

Esta separación tiene consecuencias concretas y medibles. En términos de **rendimiento**, al no existir un servidor de aplicaciones entre el usuario y el contenido estático, la latencia se reduce drásticamente; plataformas como Netlify y Vercel reportan tiempos de primer byte (TTFB) por debajo de 50 ms para sitios Jamstack distribuidos globalmente. En términos de **seguridad**, la superficie de ataque se reduce: sin servidor de aplicaciones expuesto, vulnerabilidades típicas como inyección SQL o SSRF se mitigan por diseño. En términos de **escalabilidad**, el CDN escala automáticamente ante picos de tráfico sin configuración adicional.

Frameworks representativos de este paradigma incluyen *Next.js* (React, híbrido SSR/SSG), *Nuxt* (Vue.js), *Astro* (agnóstico de framework, optimizado para contenido) y *SvelteKit*.

Sin embargo, Jamstack no es una solución universal y su adopción acrítica es un error frecuente. Aplicaciones con alta carga de datos en tiempo real — paneles de administración, sistemas de asistencia como el SGED, plataformas de comunicación — se benefician más de un backend tradicional con WebSockets o SSE (*Server-Sent Events*) que de un generador de sitios estáticos. La elección del paradigma debe responder al perfil de uso real de la aplicación, no a tendencias de mercado.

8.2 Progressive Web Apps (PWA): cierre de la brecha web/nativa

Una *Progressive Web App* es una aplicación web que aprovecha APIs modernas del navegador para ofrecer una experiencia equivalente a la de una app nativa: instalación

en pantalla de inicio, funcionamiento offline mediante *Service Workers*, notificaciones *push* y acceso a hardware del dispositivo (cámara, GPS, acelerómetro) [19].

El impacto en el contexto latinoamericano es especialmente relevante. Según datos de GSMA Intelligence, más del 60 % de los usuarios móviles en Ecuador acceden a Internet desde dispositivos de gama media-baja con conectividad intermitente. Una PWA que funcione offline y cargue en menos de 3 segundos en una red 3G tiene una ventaja competitiva sustancial frente a una app nativa que requiere descarga desde una tienda de aplicaciones y espacio de almacenamiento adicional.

El SGED ya incorpora esta tecnología: el frontend Angular incluye el paquete `@angular/service-worker` y una configuración activa en `frontend/ngsw-config.json`.

8.2.1 Demo funcional — Service Worker en el SGED

El archivo `ngsw-config.json` define dos estrategias de caché complementarias alineadas con el perfil de uso real de la aplicación:

Listing 14: Estrategias de caché del Service Worker (`frontend/ngsw-config.json`)

```
{
  "dataGroups": [
    {
      "name": "catalogos",
      "urls": ["/api/categorias/activas", "/api/estados_generales"],
      "cacheConfig": {
        "strategy": "performance",
        "maxSize": 20,
        "maxAge": "1d"
      }
    },
    {
      "name": "sesiones",
      "urls": ["/api/evaluaciones/sesion/**"],
      "cacheConfig": {
        "strategy": "freshness",
        "timeout": "3s",
```

```
        "maxSize": 30,  
        "maxAge": "12h"  
    }  
}  
],  
"navigationRequestStrategy": "performance"  
}
```

La estrategia *performance* (caché primero) aplica a los catálogos — categorías activas y estados generales — que rara vez cambian: la pantalla abre al instante aunque no haya red disponible. La estrategia *freshness* (red primero con *timeout* de 3 segundos) aplicada a los datos de evaluaciones por sesión refleja una decisión de diseño concreta: la asistencia varía durante el entrenamiento, por lo que es preferible intentar la red primero; pero si tarda más de 3 segundos — escenario habitual en canchas deportivas con cobertura débil — el Service Worker devuelve el último valor conocido en lugar de dejar al entrenador esperando con la pantalla en blanco. Esta combinación hace que el SGED funcione de forma utilizable en condiciones de conectividad intermitente sin ningún cambio en la lógica de la aplicación.

8.3 Inteligencia Artificial Generativa: reconfiguración del rol del desarrollador

La irrupción de modelos de lenguaje de gran escala (LLM) como GPT-4o, Claude 3.5 y Gemini 1.5 Pro está transformando el flujo de trabajo del desarrollador web de formas que van mucho más allá de la autocompletación de código [20].

Cuatro vectores de impacto son especialmente visibles:

1. **Generación de código y pruebas.** Herramientas como GitHub Copilot, Cursor y Tabnine generan implementaciones completas a partir de descripciones en lenguaje natural, reduciendo el tiempo de prototipado en un 40–55 % según estudios de productividad de GitHub (2023) [20].
2. **Revisión automática de código.** Modelos integrados en pipelines CI/CD identifican vulnerabilidades de seguridad, violaciones de estilo y posibles regresiones antes de que el código llegue a producción.

3. **Generación de documentación.** La especificación OpenAPI del SGED podría generarse automáticamente a partir del código fuente mediante LLMs *fine-tuned*, eliminando la brecha habitual entre código y documentación.
4. **Interfaces conversacionales.** La integración de chatbots en aplicaciones web (mediante APIs como OpenAI o Google Generative AI) permite interacciones en lenguaje natural que sustituyen formularios complejos en casos de uso de baja frecuencia.

Posición crítica. La IA generativa no elimina la necesidad de comprensión profunda del sistema. El código generado por LLMs puede contener errores sutiles, alucinaciones (referencias a funciones inexistentes) y vulnerabilidades que solo un desarrollador con criterio técnico sólido puede detectar y corregir. El riesgo real no es que la IA reemplace a los desarrolladores, sino que desarrolladores sin suficiente criterio crítico deleguen decisiones de arquitectura y seguridad a modelos que no tienen contexto del dominio de negocio. La profesión no desaparece; se desplaza hacia la validación, la especificación de requisitos precisos y el diseño de sistemas donde la IA actúa como herramienta, no como árbitro de decisiones.

En el contexto educativo ecuatoriano, esto implica que la formación en fundamentos — arquitecturas, protocolos, seguridad, bases de datos — sigue siendo indispensable: no para competir con la IA, sino para usarla con rigor y responsabilidad.

9 Seguridad, Pruebas de Rendimiento y Despliegue

9.1 Auditoría de seguridad OWASP

La auditoría no se redactó a mano: el repositorio incluye `scripts/audit-owasp.sh`, que levanta el sistema, ejecuta cada control contra la instancia real y deposita la salida cruda en `docs/mediciones/sec/`. La tabla 6 resume los controles frente al OWASP Top 10 de 2021 [21], y su columna de evidencia remite a esos archivos.

Dos filas de esa tabla merecen destacarse porque son las que un informe complaciente habría omitido. **A04 no está mitigado:** el sistema permite crear una cuenta con rol `ENTRENADOR` que no puede usar el módulo de sesiones, porque falta el registro vinculado por relación `@OneToOne` obligatoria y ningún flujo de interfaz lo crea. Es un defecto de

diseño, no un error de codificación, y por eso encaja en A04 y no en otra categoría. **A06 está parcialmente cubierto:** la versión declarada del framework figura en un CVE publicado, y el matiz de que la configuración concreta del proyecto no resulta explotable no equivale a estar al día. La acción correcta sigue siendo actualizar la dependencia.

9.2 Prueba de carga con Apache Bench

La metodología fija tres parámetros: un endpoint representativo, el número total de solicitudes (-n) y el nivel de concurrencia (-c). Se eligió GET /api/categorias/activas por ser una lectura autenticada que atraviesa la pila completa —filtro JWT, autorización por método, servicio, caché y base de datos— sin efectos secundarios que contaminen corridas sucesivas.

Listing 15: Comando ejecutado

```
ab -n 500 -c 50 -H "Authorization: Bearer $token" \
    http://localhost:8080/api/categorias/activas
```

La lectura honesta de estas cifras es que la tasa de error nula es el dato sólido y la latencia el dato mejorable: con 50 usuarios concurrentes la mediana se mantiene en 184 ms, pero la cola alta se dispara hasta 799 ms, lo que revela contención bajo carga y no un problema de capacidad bruta. Interesa señalar además un detalle metodológico documentado por quien ejecutó la prueba: una primera corrida se descartó porque el token JWT había expirado y las 500 solicitudes recibieron 401, escenario en el que ab igualmente informa un *throughput* excelente por responder errores muy rápido. Una prueba de carga sin verificar previamente que se está midiendo el recurso real puede producir números buenos y carentes de sentido.

9.3 Containerización con Docker Compose

El despliegue se orquesta con Docker Compose, que aporta tres beneficios concretos al proyecto: entornos reproducibles —el evaluador levanta exactamente la misma pila que el equipo—, aislamiento de dependencias —PostgreSQL 16 y Redis 7 conviven sin instalarse en el anfitrión— y una ruta directa a integración continua, donde el mismo archivo describe el entorno de pruebas.

La composición define cuatro servicios encadenados por dependencias con condición

de salud: `postgres` y `redis` deben reportarse `healthy` antes de que arranque `backend`, y `frontend` depende de este último. Ese encadenamiento no es cosmético: sin él, el `backend` arrancaría antes que la base y fallaría la validación del esquema. Las imágenes se fijan por digest `sha256` en lugar de por etiqueta, de modo que `postgres:16` no pueda cambiar de contenido bajo el mismo nombre entre dos despliegues.

Ninguna credencial vive en el código fuente: todas se inyectan por variables de entorno desde un archivo `.env` excluido del control de versiones, del que se versiona únicamente la plantilla `.env.example`. La razón es que el repositorio es un artefacto que se comparte, se clona y se publica, y un secreto que entra en su historial permanece en él aunque se borre en un cambio posterior. Es también la razón por la que la clave del servicio de IA quedó fuera y el módulo se deshabilita por omisión, y un argumento adicional a favor de haber elegido una API externa que no exige credencial [22].

9.4 Reflexión sobre el proceso de aprendizaje

Cada integrante consigna a continuación las lecciones técnicas que considera más relevantes del desarrollo del PFC, los errores que resultaron más costosos y qué haría de forma distinta en un proyecto futuro.

Pallo Pinto Alejandro Daniel

Mi rol en el proyecto fue seguridad y validación: el diseño de autenticación con JWT en cookie `HttpOnly`, la auditoría OWASP y la batería de pruebas. Precisamente por eso, el error que más me marcó ocurrió en mi propio terreno.

El 20 de julio moví el JWT a una cookie `HttpOnly+Secure+SameSite=Strict` y saqué los tokens del cuerpo de la respuesta del login (`bcb3642`). Cuatro días después, un *merge* del día de cierre (`ad2fec0`) reescribió el DTO y volvió a agregar los campos `accessToken` y `refreshToken`, borrando incluso el comentario que documentaba por qué no debían estar. Mi propia corrección quedó revertida sin que nadie lo advirtiera, y esa fue la versión que el docente evaluó; la rehice el 4 de agosto (`62183d8`).

Lo grave no es el *merge*, eso pasa en cualquier equipo: es que yo era el responsable de las pruebas y ninguna cubría la condición de la que dependía mi propio diseño de seguridad. La suite ya superaba el umbral de cobertura que nos habíamos fijado, y aun así nadie preguntaba lo más básico: ¿la respuesta del login trae un token?

Hoy lo haría distinto en un punto concreto: toda decisión de seguridad registrada en un ADR debería nacer junto a una prueba que falle si alguien la revierte. Un *fix* sin prueba no es un *fix*, es una intención que sobrevive hasta el siguiente *merge*. La cobertura alta me dio una confianza que no estaba justificada, porque mide cuánto código se ejecuta, no si lo que importa está protegido. La trazabilidad de este episodio queda en `OBSERVACIONES.md` bajo el código OBS-13, con los tres *commits* implicados.

Beltrán

Pendiente de redacción por el integrante (mínimo 100 palabras).

Cruz

Pendiente de redacción por el integrante (mínimo 100 palabras).

10 Conclusiones

1. **REST como estándar consolidado.** La disertación de Fielding [1] sigue siendo el fundamento teórico irrefutable de las APIs modernas. El SGED implementa sus restricciones de forma coherente, lo que se traduce en una API comprensible, extensible y documentada mediante OpenAPI 3.1 / Swagger UI.
2. **SOAP: contexto, no obsolescencia.** SOAP coexiste con REST en entornos donde la integridad del mensaje a nivel protocolar es crítica (banca, servicios gubernamentales). En Ecuador, la transición en el sector público avanza lentamente por razones de compatibilidad histórica [6], lo que hace relevante conocer ambos paradigmas.
3. **MVC como arquitectura estructurante.** El patrón MVC, implementado con Spring Boot y Angular, demostró ser suficientemente flexible para soportar los módulos de asistencia QR, gestión de categorías y control de usuarios del SGED, manteniendo una separación clara de responsabilidades.
4. **Tendencias como oportunidad, no amenaza.** Jamstack, PWA e IA generativa complementan y amplifican las competencias técnicas fundamentales. Un equipo

que entiende los protocolos subyacentes puede aprovechar estas tecnologías con criterio; uno que no los entiende, construye sobre cimientos que no comprende.

5. **Documentación como ciudadano de primera clase.** La integración de SpringDoc en el SGED convirtió la documentación en una consecuencia automática del código (*doc-as-code*), reduciendo la divergencia entre la API real y su descripción formal.

11 Trabajo Futuro

- **PWA — registro offline de asistencias:** el SGED ya cuenta con un Service Worker activo (`ngsw-config.json`) orientado a caché de catálogos y sesiones. El siguiente paso es extenderlo con una cola de escritura offline que permita registrar asistencias sin conexión y sincronizarlas con el servidor al recuperar señal, completando así el ciclo PWA para el flujo de entrenamiento.
- **GraphQL como capa alternativa:** exponer un endpoint GraphQL sobre los mismos servicios del SGED para permitir consultas flexibles desde el frontend sin *over-fetching* ni *under-fetching*.
- **Generación de pruebas con IA:** integrar un agente basado en LLM que, a partir de la especificación OpenAPI, genere automáticamente casos de prueba de integración para el backend.
- **Migración a OAuth2 / OIDC:** reemplazar el JWT artesanal actual por un proveedor de identidad estándar (Keycloak, Auth0) para facilitar la integración con sistemas universitarios externos.
- **Análisis de rendimiento Jamstack:** prototipar el frontend con Next.js en modo SSG/ISR para comparar métricas de Core Web Vitals contra la versión SPA actual.

Referencias

- [1] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures”, Tesis doct., University of California, Irvine, 2000. dirección: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [2] SpringDoc Team. “SpringDoc OpenAPI 2 — Reference Documentation”, visitado 1 de ago. de 2026. dirección: <https://springdoc.org/v2/>
- [3] L. Richardson y S. Ruby, *RESTful Web Services*. Sebastopol, CA: O’Reilly Media, 2007, ISBN: 978-0-596-52926-0.
- [4] W3C, “SOAP Version 1.2 Part 1: Messaging Framework (Second Edition)”, World Wide Web Consortium, inf. téc., 2007. dirección: <https://www.w3.org/TR/soap12-part1/>
- [5] M. Masse, *REST API Design Rulebook*. Sebastopol, CA: O’Reilly Media, 2011, ISBN: 978-1-4493-1050-9.
- [6] M. Garriga, C. Mateos, A. Flores, A. Cechich y A. Zunino, “RESTful service composition at a glance: A survey”, *Journal of Network and Computer Applications*, vol. 60, págs. 32-53, 2021. DOI: [10.1016/j.jnca.2015.11.020](https://doi.org/10.1016/j.jnca.2015.11.020)
- [7] R. T. Fielding y J. F. Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content”, Internet Engineering Task Force (IETF), inf. téc. RFC 7231, 2014. dirección: <https://www.rfc-editor.org/rfc/rfc7231>
- [8] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.2”, W3C Recommendation, inf. téc., 2023. dirección: <https://www.w3.org/TR/WCAG22/>
- [9] International Organization for Standardization, “ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model”, ISO/IEC, inf. téc., 2023.
- [10] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley, 2003, ISBN: 978-0-321-12521-7.
- [11] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O’Reilly Media, 2017, ISBN: 978-1-4493-7332-0.

- [12] T. Reenskaug, “Thing-Model-View-Editor: An Example from a Planning System”, Xerox Palo Alto Research Center, inf. téc., 1979.
- [13] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.
- [14] TechEmpower. “Framework Benchmarks Round 22”, visitado 11 de ago. de 2026. dirección: <https://www.techempower.com/blog/2023/11/15/framework-benchmarks-round-22/>
- [15] M. Nottingham y E. Wilde, “RFC 7807: Problem Details for HTTP APIs”, Internet Engineering Task Force, inf. téc., 2016. dirección: <https://www.rfc-editor.org/rfc/rfc7807>
- [16] Open-Meteo. “Free Weather Forecast API — Documentation”, visitado 11 de ago. de 2026. dirección: <https://open-meteo.com/en/docs>
- [17] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2.^a ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.
- [18] M. Biilmann. “The JAMstack origin story”, visitado 1 de ago. de 2026. dirección: <https://www.netlify.com/blog/2020/04/14/what-is-a-static-site-generator/>
- [19] MDN Web Docs. “Progressive web apps (PWAs)”, visitado 1 de ago. de 2026. dirección: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
- [20] GitHub, “The economic impact of the AI-powered developer lifecycle”, GitHub, Inc., inf. téc., 2023. dirección: <https://github.blog/2023-06-27-the-economic-impact-of-the-ai-powered-developer-lifecycle/>
- [21] OWASP Foundation. “OWASP Top 10:2021”, visitado 11 de ago. de 2026. dirección: <https://owasp.org/Top10/>
- [22] H. Washizaki, ed., *Guide to the Software Engineering Body of Knowledge (SWE-BOK), Version 4.0*. Piscataway, NJ, USA: IEEE Computer Society, 2024.

A Anexo C — Preguntas de Análisis

A.1 La decisión técnica más difícil (Bloom 6 — Evaluación)

Revisando el PFC completo, la decisión más difícil no fue la elección del framework —acotada por el docente y resuelta en el ADR-001— sino **dónde almacenar el JWT en el cliente**, registrada en los ADR-002 y ADR-007.

Se consideraron dos alternativas. La primera, `localStorage` con cabecera `Authorization: Bearer`, es la más difundida en tutoriales y la más simple de programar: el frontend controla el token explícitamente y no hay que lidiar con CORS con credenciales ni con `SameSite`. Su defecto es estructural: cualquier XSS con ejecución de JavaScript lee `localStorage` y exfiltra la sesión completa. La segunda, cookie `HttpOnly + Secure + SameSite=Strict`, vuelve el token ilegible para JavaScript, de modo que un XSS deja de ser automáticamente una toma de sesión, pero introduce tres costos reales: obliga a `withCredentials: true` y a una configuración CORS más estricta, complica las pruebas manuales —el token no se puede copiar y pegar— y traslada al servidor el problema de la revocación, porque una cookie no se puede “borrar” de forma fiable desde el servidor.

Se eligió la cookie `HttpOnly`, con una lista negra de revocación en Redis para el cierre de sesión. Con el conocimiento actual, **se tomaría la misma decisión**, y el criterio que la sostiene es de contexto y no de moda: el sistema maneja datos de menores de edad —nombres, cédulas, lesiones—, y en ese escenario el impacto de una sesión robada pesa más que la comodidad de desarrollo. El propio informe aporta evidencia a favor: la auditoría A05 confirma una CSP restrictiva, y ambas medidas se refuerzan mutuamente. Lo que sí se haría distinto es el orden: la revocación en Redis se agregó después, y hubo un período en que cerrar sesión no invalidaba realmente el token.

A.2 Escalar a 500 usuarios concurrentes con 0 % de error (Bloom 5 — Síntesis)

La medición de partida es concreta: **211,33 req/s con 0 % de error** a concurrencia 50, pero con P95 de 597 ms y P99 de 701 ms. El punto importante es que la tasa de error ya es cero; el problema al multiplicar por diez la carga no sería de corrección sino de saturación, y la cola alta de latencia indica dónde.

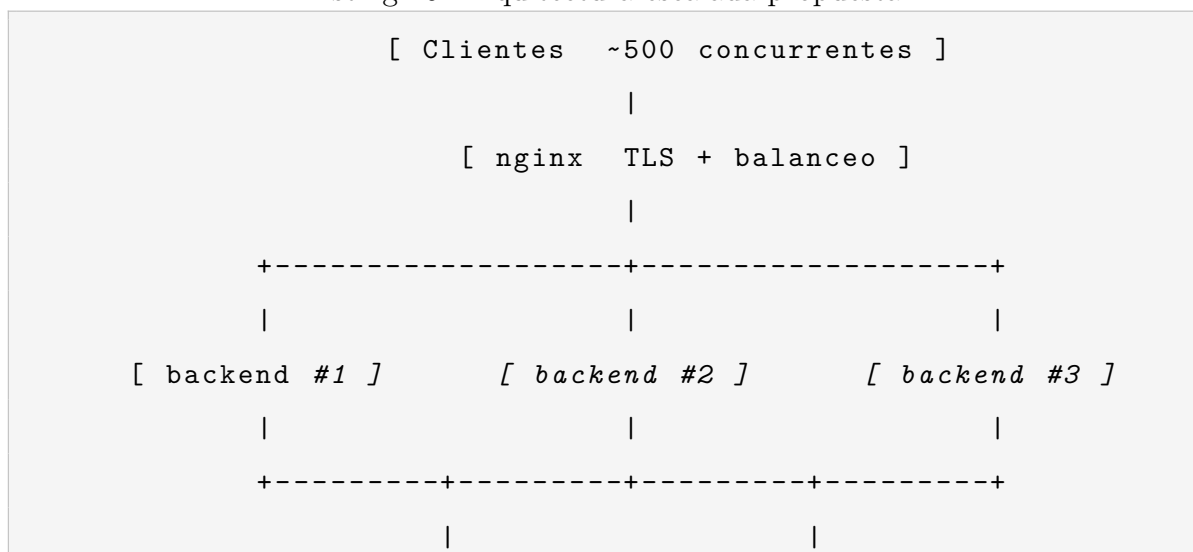
La hipótesis principal es el **pool de conexiones**. El proyecto no configura HikariCP de forma explícita, por lo que opera con el valor por omisión de 10 conexiones: con 50

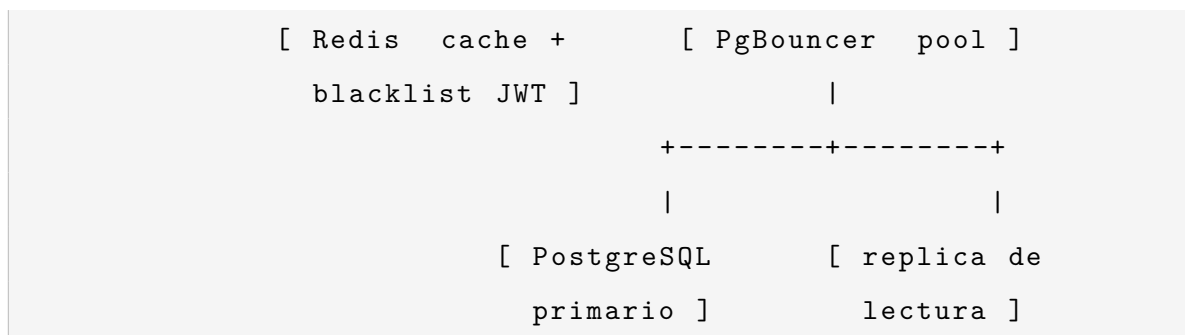
peticiones concurrentes, 40 esperan turno, y eso explica una mediana de 184 ms junto a un máximo de 799 ms mucho mejor que suponer lentitud de las consultas. El primer cambio, por tanto, es medir antes de escalar: instrumentar el tiempo de espera del pool y contrastarlo con el tiempo real de consulta.

Los cambios necesarios, en orden de costo-beneficio:

1. **Dimensionar el pool y la base.** Subir `maximum-pool-size` de forma proporcional al paralelismo real de PostgreSQL, e introducir PgBouncer si el número de conexiones supera lo que el motor sostiene con holgura.
2. **Replicar el backend horizontalmente.** Es viable porque la autenticación es *stateless* vía JWT y la caché es externa en Redis: varias réplicas detrás del nginx que ya existe no necesitan sesión pegajosa. La lista negra de revocación, compartida en Redis, sigue siendo coherente entre réplicas.
3. **Habilitar caché HTTP en las lecturas cacheables.** Hoy las respuestas viajan con `Cache-Control: no-store` por la configuración de Spring Security, lo que impide cualquier caché intermedia (véase el análisis del principio *cacheable* en la pregunta siguiente). Los catálogos —categorías, estados— podrían servirse con ETag y validación condicional, retirando carga del origen sin tocar la base.
4. **Réplicas de lectura en PostgreSQL** para separar las consultas de informes de la ruta transaccional.

Listing 16: Arquitectura escalada propuesta





Sostener 0 % de error a esa escala exige además dos piezas que hoy no existen: un *circuit breaker* en las llamadas al proveedor externo de clima y límites de concurrencia por endpoint, para que una saturación degrade servicio en lugar de encolar hasta el agotamiento [17].

A.3 Los 6 principios REST de Fielding frente a la API del SGED (Bloom 4 — Análisis)

El hallazgo con mayor consecuencia práctica es el tercero: el SGED se describe como una API REST cacheable y sus cabeceras dicen literalmente lo contrario. No es un defecto de implementación sino una omisión de diseño —nadie revisó la configuración por omisión de Spring Security frente al principio— y es también la palanca más barata para escalar, según se argumentó en la pregunta anterior.

A.4 Comparación con un sistema de producción y calidad ISO/IEC 25010 (Bloom 6)

El equipo ya mantiene una matriz de atributos de calidad con catorce escenarios trazados a evidencia ejecutable (`docs/iso25010-atributos-calidad.md`), lo que permite responder con datos propios y no con impresiones. Contrastado con lo que ofrece una plataforma comercial de gestión de clubes deportivos, el balance es el siguiente.

Cubierto de forma verificable: seguridad (control de acceso por rol, credenciales con BCrypt de factor 12, TLS y cabeceras endurecidas, todo con evidencia en `docs/mediciones/sec/`); mantenibilidad (organización por dominios, cobertura JaCoCo con umbral en `pom.xml`, 211 pruebas); portabilidad (arranque reproducible con imágenes fijadas por digest); e interoperabilidad (OpenAPI y colección Postman versionada).

Requiere trabajo adicional para ser comercialmente viable: la *fiabilidad*,

porque no hay estrategia de respaldo y restauración de la base ni despliegue redundante—un fallo del único contenedor de PostgreSQL implica pérdida de datos—; la *eficiencia de desempeño* bajo carga real, según la cola de latencia ya discutida; la *funcionalidad*, con dos brechas ya documentadas por el propio equipo (la fila 14 de su matriz y el hallazgo A04 sobre el alta de entrenadores); y la *usabilidad accesible*, donde existe configuración de Lighthouse pero no una auditoría WCAG sistemática.

Una estimación honesta para cerrar esas brechas —respaldos y recuperación, alta disponibilidad, corrección de A04, ajuste de rendimiento y auditoría de accesibilidad— es de **ocho a doce semanas** de un equipo de tres personas a dedicación parcial, sin contar los requisitos legales de tratamiento de datos de menores, que en un producto comercial serían condición previa y no un atributo más.

Nota de alcance: esta comparación se hace contra las características públicamente documentadas de la categoría de plataformas de gestión deportiva, no contra un caso de estudio publicado en particular. Si la evaluación exige citar uno específico, corresponde al equipo seleccionarlo e incorporarlo a las referencias; no se cita aquí ninguno para no atribuir datos a una fuente que no se consultó directamente.

B Archivo `referencias.bib`

A continuación se incluye el contenido completo del archivo `referencias.bib` que debe existir en la misma carpeta que este documento para que la compilación con `biber` funcione correctamente.

```
@phdthesis{fielding2000,
  author      = {Fielding, Roy Thomas},
  title       = {Architectural Styles and the Design of
                 Network-based Software Architectures},
  school      = {University of California, Irvine},
  year        = {2000},
  url         = {https://ics.uci.edu/~fielding/pubs/dissertation/top.htm}
}

@book{richardson2007,
  author      = {Richardson, Leonard and Ruby, Sam},
  title       = {RESTful Web Services},
```

```
    publisher = {O'Reilly Media},
    year      = {2007},
    isbn      = {978-0-596-52926-0}
}

@online{springdoc2024,
  author = {{SpringDoc Team}},
  title  = {SpringDoc OpenAPI 2 --- Reference Documentation},
  year   = {2024},
  url    = {https://springdoc.org/v2/},
  urldate = {2026-08-01}
}

@techreport{w3c2007,
  author      = {{W3C}},
  title       = {SOAP Version 1.2 Part 1: Messaging Framework
                (Second Edition)},
  institution = {World Wide Web Consortium},
  year        = {2007},
  url         = {https://www.w3.org/TR/soap12-part1/}
}

@book{masse2011,
  author = {Masse, Mark},
  title  = {REST API Design Rulebook},
  publisher = {O'Reilly Media},
  year    = {2011},
  isbn    = {978-1-4493-1050-9}
}

@article{garriga2021,
  author = {Garriga, Martin and Mateos, Cristian and
            Flores, Alejandro and Cechich, Alejandra and
            Zunino, Alejandro},
  title  = {RESTful service composition at a glance:
            A survey},
  journal = {Journal of Network and Computer Applications},
  year    = {2021},
  volume  = {60},
```

```
pages    = {32--53},
doi      = {10.1016/j.jnca.2015.11.020}
}

@online{biilmann2020,
  author   = {Biilmann, Mathias},
  title    = {The JAMstack origin story},
  year     = {2020},
  url      = {https://www.netlify.com/blog/2020/04/14/
              what-is-a-static-site-generator/},
  urldate  = {2026-08-01}
}

@online{mozilla2023,
  author   = {{MDN Web Docs}},
  title    = {Progressive web apps (PWAs)},
  year     = {2023},
  url      = {https://developer.mozilla.org/en-US/docs/Web/
              Progressive_web_apps},
  urldate  = {2026-08-01}
}

@techreport{github2023,
  author    = {{GitHub}},
  title     = {The economic impact of the AI-powered
              developer lifecycle},
  institution = {GitHub, Inc.},
  year      = {2023},
  url       = {https://github.blog/2023-06-27-the-economic-impact-
              of-the-ai-powered-developer-lifecycle/}
}
```

Cuadro 6: Controles OWASP en el SGED: vector, contramedida y evidencia

Riesgo	Vector en el SGED	Contramedida implementada	Evidencia
A01 Control de acceso roto	Un ENTRENADOR invoca un recurso de ADMINISTRADOR	@PreAuthorize por método sobre los seis roles; matriz completa verificada	a01-acceso-roto.txt, a01-roles-nuevos.txt
A02 Fallos criptográficos	Sesión o credenciales viajando en claro	TLS terminado en nginx; JWT en cookie <i>HttpOnly</i> ; HSTS <i>max-age=31536000</i>	a02-tls.txt
A03 Inyección	' OR '1'='1 en campos de alta de estudiante	Consultas parametrizadas vía JPA y procedimientos almacenados; validación con <i>Bean Validation</i> → 422	a03-inyeccion.txt
A04 Diseño inseguro	Alta de usuario ENTRENADOR sin registro en <i>deportivo.entrenadores</i> : estado de negocio inválido alcanzable	No mitigado. Detectado en revisión entre pares (REV-05)	REVISION-BELTRAN.md
A05 Configuración insegura	Cabeceras ausentes, <i>clickjacking</i> , <i>MIME sniffing</i>	CSP default-src 'self'; frame-ancestors 'none', X-Frame-Options: DENY, X-Content-Type-Options: nosniff	a05-cabeceras.txt
A06 Componentes vulnerables	spring-boot-started fijado en 3.2.5	Parcial CVE-2025-22235 aplica a la versión, pero no es explotable aquí porque <i>SecurityConfig</i> protege <i>/actuator/*</i> con rutas literales y no con <i>EndpointRequest.to()</i>	REVISION-BELTRAN.md (REV-08)
A07 Fallos de autenticación	Fuerza bruta sobre POST <i>/api/auth/login</i>	Bloqueo por IP tras 5 intentos en ventana de 15 min → 429 con <i>Problem Details</i>	a07-rate-limit.txt
A09 Fallos de registro	Ataque de credenciales sin rastro auditable	<i>Logger</i> AUTH_AUDIT con IP, marca temporal y sujeto en cada AUTH_LOGIN_OK / AUTH_LOGIN_FAIL	a09-logging.txt
XSS	Inyección de <i>script</i> vía campos de texto mostrados en la	Escapado automático de Angular por omisión más CSP restrictiva; el	a05-cabeceras.txt

Cuadro 7: Resultados de Apache Bench (500 solicitudes, concurrencia 50)

Métrica	Valor
Solicitudes completadas	500
Solicitudes fallidas	0
Rendimiento (<i>throughput</i>)	211,33 req/s
Tiempo total de la prueba	2,366 s
P50 (mediana)	184 ms
P95	597 ms
P99	701 ms
Máximo	799 ms

Cuadro 8: Cumplimiento de los principios REST [1] en la API del SGED

Principio	Cumple	Análisis y costo de no cumplirlo
Cliente–Servidor	Completo	SPA Angular y backend Spring Boot son artefactos desplegables independientes que solo se comunican por HTTP/JSON.
Sin estado	Parcial	La identidad viaja en el JWT y no hay sesión HTTP en el servidor. Sin embargo, la revocación consulta una lista negra en Redis, de modo que validar un token exige estado compartido. <i>Costo</i> : cada réplica depende de Redis; sin él no se puede distinguir un token revocado de uno vigente. Se aceptó a cambio de que cerrar sesión sea efectivo.
Cacheable	No cumplido	Las respuestas salen con Cache-Control: no-cache, no-store, must-revalidate (verificable en <code>a05-cabeceras.txt</code>), configuración por omisión de Spring Security para endpoints autenticados. <i>Costo</i> : ninguna caché de navegador, proxy o CDN puede intervenir, y catálogos que casi no cambian se recalculan en cada petición. Se compensa con caché de servidor en Redis, que reduce la carga de base pero no el tráfico de red.
Interfaz uniforme	Parcial	URIs con sustantivos en plural (<code>/api/categorias</code> , <code>/api/estudiantes</code>), verbos HTTP semánticos y códigos correctos (201, 204, 404, 422). Pero no hay hipermedia: la API se sitúa en el nivel 2 del modelo de madurez de Richardson, no en el 3. Persisten además endpoints con verbo en la ruta, como <code>/api/estudiantes/operaciones/desactivar-categoria</code> o <code>/api/consentimientos/{id}/revocar</code> . <i>Costo</i> : el cliente debe conocer de antemano todas las rutas y su acoplamiento al servidor es mayor; para un frontend propio y versionado junto al backend, el costo es bajo y consciente.
Sistema por capas	Completo	nginx termina TLS delante del backend y el cliente no distingue si habla con el proxy o con la aplicación; Redis y PgBouncer serían capas igualmente invisibles.
Código bajo demanda	No aplica	Principio explícitamente opcional en la formulación de Fielding. El frontend se entrega como paquete estático compilado. <i>Costo</i> : ninguno.